Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE1113 Sistemas Empotrados
Prof. Dr.-Ing. Jeferson González Gómez

Taller 2: GCC

Etapas de compilación y bibliotecas

1. Introducción

Todo el software que se ejecuta en un sistema empotrado, ya sea el kernel de Linux, sus controladores o la aplicación final, necesita ser traducido de código fuente en C a instrucciones que el procesador pueda ejecutar. En el mundo del software libre, esa traducción la realiza **GCC** (*GNU Compiler Collection*), el compilador estándar de facto en sistemas Linux y en la mayoría de *toolchains* de cruce (*cross-compilation*) usados para arquitecturas embebidas como ARM o RISC-V.

Comprender **qué hace GCC internamente** —y no solo cómo invocarlo con sus opciones más comunes— es lo que permite luego automatizar la construcción con herramientas como Make (taller 3) y diagnosticar problemas de compilación o enlazado en proyectos reales. Este taller recorre las cuatro etapas de compilación, la diferencia entre bibliotecas estáticas y dinámicas, y las herramientas del *toolchain* de GNU que acompañan a GCC. Al finalizar, el estudiante aplicará estos conocimientos construyendo una biblioteca en C directamente con `gcc`, sin apoyo de un sistema de construcción automatizado.

2. Fundamentos de GCC

2.1. ¿Qué es GCC?

GCC (*GNU Compiler Collection*) es un conjunto de compiladores del proyecto GNU que soporta múltiples lenguajes (C, C++, Fortran, Ada, entre otros) y múltiples arquitecturas de destino. Cuando se invoca el comando `gcc` sobre un archivo `.c`, en realidad se está orquestando una serie de programas independientes que se ejecutan en cadena, cada uno especializado en una tarea distinta. Esa cadena completa —desde el código fuente hasta el ejecutable final— se conoce como **toolchain**.

2.2. Las cuatro etapas de compilación

Aunque el comando `gcc archivo.c -o programa` parece un solo paso, internamente se ejecutan cuatro etapas independientes:

1. **Preprocesado**: el **preprocesador de C** (`cpp`) expande directivas que comienzan con `#` — `#include`, `#define`, `#ifdef`— sustituyendo macros e insertando el contenido de los archivos de cabecera. El resultado es un único archivo de código fuente "aplanado", sin directivas de preprocesador.

2. **Compilación**: el compilador propiamente dicho (`cc1` para C) traduce el código fuente preprocesado a código en **lenguaje ensamblador** para la arquitectura destino, aplicando además las optimizaciones solicitadas.

3. **Ensamblado**: el **ensamblador** (`as`) traduce el código en lenguaje ensamblador a **código objeto** (formato binario ELF, `.o`), que contiene instrucciones de máquina pero todavía referencias sin resolver a símbolos externos.

4. **Enlazado**: el **enlazador** (`ld`) combina uno o más archivos objeto con las bibliotecas necesarias, resuelve las referencias entre símbolos y produce el **ejecutable** final (o una biblioteca).

GCC permite detener el proceso después de cualquiera de estas etapas e inspeccionar el resultado intermedio, lo cual es sumamente útil para depurar problemas de compilación. Usando como ejemplo un archivo `saludo.c`:

Etapa	Bandera	Comando y salida
Preprocesado	`-E`	`gcc -E saludo.c -o saludo.i`
Compilación	`-S`	`gcc -S saludo.c -o saludo.s`
Ensamblado	`-c`	`gcc -c saludo.c -o saludo.o`
Enlazado	*(ninguna)*	`gcc saludo.c -o saludo`

GCC no ejecuta estas etapas él mismo: internamente invoca programas independientes (`cc1`, `as`, `collect2`/`ld`) encadenados en un solo comando. Para observar esta cadena real de invocaciones puede usarse la bandera `-v` (*verbose*), que imprime cada comando ejecutado internamente antes de correrlo:

```
gcc -v -Wall saludo.c -o saludo
```

Entre la salida (abreviada aquí para conservar únicamente las banderas más relevantes de cada invocación) aparecen tres programas encadenados, cada uno tomando como entrada el archivo temporal que produjo el anterior. En los listados siguientes, `...` **no** es parte del comando real: es una elipsis que indica rutas y banderas adicionales omitidas por brevedad (la salida real de `cc1` y `collect2` incluye muchas más rutas de búsqueda y opciones de las mostradas aquí).

1. **Preprocesado + compilación** (`cc1`): recibe directamente el código fuente y produce ensamblador en un archivo temporal `.s`:

```
/usr/libexec/gcc/x86_64-linux-gnu/13/cc1 -quiet saludo.c \
  -mtune=generic -march=x86-64 -Wall -o /tmp/cc15uSsx.s
```

   Nótese que `cc1` realiza el preprocesado y la compilación en un solo paso interno: no se genera un archivo `.i` intermedio en disco a menos que se invoque `gcc -E` explícitamente.

2. **Ensamblado** (`as`): toma el archivo `.s` que generó `cc1` y produce el archivo objeto `.o`:

```
as -v --64 -o /tmp/ccmnrcVU.o /tmp/cc15uSsx.s
```

3. **Enlazado** (`collect2`, que a su vez invoca a `ld`): combina el objeto anterior con los objetos de arranque de C (`crt1.o`, `crti.o`, `crtn.o`) y bibliotecas del sistema (`-lc`, `-lgcc`) para producir el ejecutable final:

```
/usr/libexec/gcc/x86_64-linux-gnu/13/collect2 ... -o saludo \
  .../Scrt1.o .../crti.o .../crtbeginS.o /tmp/ccmnrcVU.o \
  -lgcc -lgcc_s -lc -lgcc -lgcc_s .../crtendS.o .../crtn.o
```

Obsérvese la cadena de archivos temporales: la salida de `cc1` (`cc15uSsx.s`) es la entrada de `as`, y la salida de `as` (`ccmnrcVU.o`) es uno de los objetos que recibe `collect2`. Esa cadena de entrada/salida es la evidencia concreta de que las cuatro etapas descritas no son una simplificación pedagógica: son, literalmente, los programas independientes que `gcc` orquesta —y encadena mediante archivos temporales— en cada compilación. La bandera relacionada `-###` muestra los mismos comandos sin ejecutarlos, útil para inspeccionar la cadena sin generar archivos.

2.3. Banderas comunes de GCC

`-o <archivo>`	Nombre del archivo de salida (en cualquier etapa).
`-Wall`	Habilita advertencias comunes del compilador; se recomienda usarla siempre para detectar errores potenciales.
`-g`	Incluye información de depuración (símbolos, líneas de código fuente) para poder usar GDB.
`-O0`, `-O1`, `-O2`, `-O3`	Nivel de optimización del código generado; `-O0` (sin optimizar) es la más adecuada para depurar.
`-I<directorio>`	Agrega un directorio a la búsqueda de archivos de cabecera (`#include`).
`-L<directorio>`	Agrega un directorio a la búsqueda de bibliotecas en tiempo de enlazado.
`-l<nombre>`	Enlaza contra la biblioteca `lib<nombre>` (por ejemplo `-lm` enlaza `libm`, la biblioteca matemática).
`-fPIC`	Genera código independiente de la posición de carga en memoria (*Position Independent Code*), requerido para bibliotecas dinámicas.
`-shared`	Indica al enlazador que produzca una biblioteca dinámica en lugar de un ejecutable.
`-static`	Fuerza el enlazado estático de todas las bibliotecas.

2.4. Bibliotecas estáticas y dinámicas

Una **biblioteca** es una colección de funciones ya compiladas que pueden reutilizarse en distintos programas sin volver a compilar su código fuente. En Linux existen dos tipos:

- **Biblioteca estática** (`.a`): es, en esencia, un archivo que agrupa varios archivos objeto (`.o`) mediante la herramienta `ar`. Cuando un programa se enlaza contra una biblioteca estática, el código de las funciones utilizadas se **copia dentro del ejecutable final**. Esto produce binarios más grandes y autocontenidos, que no dependen de que la biblioteca esté presente en el sistema donde se ejecuten.
- **Biblioteca dinámica** o **compartida** (`.so`, *shared object*): el código de la biblioteca **permanece separado** del ejecutable y se carga en memoria en tiempo de ejecución. Varios programas pueden compartir una misma copia de la biblioteca cargada en memoria, lo cual ahorra espacio en disco y en RAM, y permite actualizar la biblioteca sin recompilar los programas que la usan. A cambio, el sistema debe poder **encontrar** el archivo `.so` en tiempo de ejecución, ya sea en una ruta estándar (`/usr/lib`) o en una ruta indicada mediante la variable de entorno `LD_LIBRARY_PATH`.

Para compilar código fuente que sea apto para una biblioteca compartida es necesario usar `-fPIC`; para producir el archivo `.so` propiamente dicho se usa `-shared`. Por ejemplo, a partir de un archivo objeto `modulo.o` compilado con `-fPIC`:

```
# Biblioteca estatica
ar rcs libmodulo.a modulo.o

# Biblioteca dinamica
gcc -shared -o libmodulo.so modulo.o
```

2.5. Herramientas del *toolchain*

Además de `gcc`, el paquete **binutils** de GNU incluye otras herramientas de línea de comandos útiles para trabajar con archivos objeto, bibliotecas y ejecutables:

`ar`	Crea y manipula archivos de biblioteca estática (`.a`).
`objcopy`	Copia y transforma un archivo objeto o ejecutable entre formatos (por ejemplo, de ELF a binario puro), útil para generar imágenes de firmware a partir de un ejecutable ELF.
`nm`	Lista los símbolos (funciones y variables) definidos y referenciados en un archivo objeto o biblioteca.
`objdump`	Desensambla y muestra información detallada de un archivo objeto o ejecutable (`objdump -d` para ver el desensamblado).
`addr2line`	Traduce una dirección de memoria a su archivo fuente y número de línea, usando los símbolos de depuración generados con `-g`; útil para interpretar volcados de pila.
`readelf`	Muestra la estructura interna de un archivo en formato ELF (cabeceras, secciones, símbolos).
`ldd`	Lista las bibliotecas dinámicas de las que depende un ejecutable y dónde las encuentra el sistema.
`strip`	Elimina símbolos y otra información de depuración de un binario, reduciendo su tamaño.
`size`	Reporta el tamaño de las secciones (código, datos) de un archivo objeto o ejecutable.

Por ejemplo, para verificar de qué bibliotecas dinámicas depende un ejecutable enlazado dinámicamente:

```
ldd programa
```

2.6. Ejemplo integrador

El siguiente ejemplo construye, únicamente con `gcc` y `ar` (sin ningún sistema de construcción automatizado), una biblioteca mínima *libhello* con una sola función, en sus dos variantes. El código fuente de referencia es el siguiente:

say.h

```
#ifndef SAY_H
#define SAY_H

void say_hello(void);

#endif /* SAY_H */
```

say.c

```
#include "say.h"
#include <stdio.h>

void say_hello(void)
{
    printf("Hello, World!\n");
}
```

main.c

```c
#include "say.h"

int main(void)
{
    say_hello();
    return 0;
}
```

Con estos tres archivos en el mismo directorio, los siguientes comandos construyen y prueban ambas variantes de la biblioteca:

```
# 1. Compilar el objeto de la biblioteca, con -fPIC para permitir su
#    uso tanto en la version estatica como en la dinamica
gcc -Wall -fPIC -c say.c -o say.o

# 2. Biblioteca estatica: empaquetar el objeto con ar
ar rcs libhello.a say.o

# 3. Biblioteca dinamica: enlazar el objeto como shared object
gcc -shared -o libhello.so say.o

# 4. Programa que usa la biblioteca, enlazado ESTATICAMENTE
gcc -Wall main.c libhello.a -o saludo_e

# 5. Programa que usa la biblioteca, enlazado DINAMICAMENTE
gcc -Wall main.c -L. -lhello -o saludo_d

# 6. Para ejecutar la variante dinamica, el sistema debe encontrar
#    libhello.so en tiempo de ejecucion
export LD_LIBRARY_PATH=$(pwd):$LD_LIBRARY_PATH
./saludo_d
```

Nótese que en el paso 5 la bandera `-lhello` le indica al enlazador que busque `libhello.so` (o, en su ausencia, `libhello.a`) en los directorios indicados con `-L`; el prefijo `lib` y la extensión se agregan automáticamente por convención.

3. Evaluación

3.1. Preguntas teóricas

1. ¿Qué es GCC?
2. ¿Cuáles son las 4 etapas de compilación?
3. ¿Qué comando debería utilizar para generar el código en ensamblador de un archivo fuente, por ejemplo, `calculadora.c`?
4. ¿Cuál es la diferencia entre una biblioteca estática y una dinámica?

3.2. Ejercicio 1: Construcción de la biblioteca

Descripción
Debe crear una biblioteca, en lenguaje C, la cual ofrecerá cinco funciones matemáticas: suma, resta, multiplicación, división y raíz cuadrada. Para el desarrollo de la biblioteca, tome en cuenta los siguientes aspectos:

- Debe crear un archivo de cabecera `biblioteca.h` (los nombres de archivos son genéricos, puede cambiarlos a conveniencia), que contenga únicamente la **definición**/prototipo de las funciones a utilizar y las variables globales que requiera. Debe crear además el archivo `biblioteca.c`, donde desarrolle cada una de las funciones.
- La biblioteca deberá ser creada tanto estática como dinámicamente, es decir, al finalizar el ejercicio deberá tener dos archivos: `libbiblioteca.a` y `libbiblioteca.so`. La construcción debe realizarse invocando `gcc` y `ar` directamente (no utilice Make ni ninguna otra herramienta de construcción automatizada).
- Por último, debe crear un archivo `calculadora.c` y generar a partir de él **dos ejecutables**: `calculadora_e` (enlazado estáticamente contra `libbiblioteca.a`) y `calculadora_d` (enlazado dinámicamente contra `libbiblioteca.so`), para verificar el correcto funcionamiento de ambas variantes de la biblioteca.

3.3. Ejercicio 2: Inspección del binario con herramientas del *toolchain*

Descripción
A partir de los archivos generados en el Ejercicio 1, utilice las herramientas del *toolchain* descritas en la sección de teoría (`size`, `nm`, `ldd`, `objdump`, `strip`) para responder, con evidencia de los comandos ejecutados y su salida, las siguientes preguntas:

- Compare el tamaño en disco de `calculadora_e` y `calculadora_d` con `ls -lh` y el tamaño de sus secciones internas con `size`. ¿Cuál es mayor y por qué?
- Utilice `nm` sobre `libbiblioteca.a` para listar los símbolos de las cinco funciones matemáticas. Identifique, según el tipo de símbolo reportado por `nm`, cuáles están definidos en ese archivo.
- Ejecute `ldd` sobre ambos ejecutables. ¿Aparece `libbiblioteca.so` como dependencia en los dos casos? Explique la diferencia observada en términos de cómo se enlazó cada uno.

- Utilice `objdump -d` sobre `calculadora_d` para localizar, dentro del desensamblado, la instrucción donde se invoca una de las cinco funciones matemáticas.
- Genere una copia de `calculadora_d` sin símbolos de depuración usando `strip`, y repita el comando `nm` sobre ambas versiones (con y sin `strip`). ¿Qué diferencia observa en la salida?

3.4. Entregable

- Archivo `.tar` con los archivos: `biblioteca.h`, `biblioteca.c`, `libbiblioteca.a`, `libbiblioteca.so`, `calculadora.c`, `calculadora_e` y `calculadora_d`, además de un archivo `.pdf` con la solución a las preguntas teóricas, los comandos necesarios para la construcción de ambas bibliotecas (Ejercicio 1) y las respuestas del Ejercicio 2 con la evidencia de cada comando ejecutado.

Material de referencia

- GCC — Using the GNU Compiler Collection: `https://gcc.gnu.org/onlinedocs/gcc/`
- GNU Binutils: `https://www.gnu.org/software/binutils/`
- Program Library HOWTO: `https://tldp.org/HOWTO/Program-Library-HOWTO/`
- Aguilar Ulloa, M. *Tutorial 4: GCC*. Material del curso, ITCR.